

Unmasking the Board

Behavioral Anomaly Detection in Human Chess

Christoph · Kiril · Ali · Georgy

BCSAI — Machine Learning Foundations | IE University | 2026

github.com/Kiril-P/Final-Group-Project-Machine-Learning | Dataset: July 2016 Lichess Export

We apply unsupervised anomaly detection to identify statistically unusual player behavior in online chess. Using 500,000 games sampled from the July 2016 Lichess export (17,909 unique players), we engineer 21 features per player (8 behavioral and 13 chess-accuracy-derived) and compare seven detectors: a Z-Score baseline, Local Outlier Factor (LOF), Isolation Forest, One-Class SVM, an Autoencoder, HDBSCAN, and a chess-accuracy-focused sub-autoencoder. Evaluated against synthetic anomaly injection across three difficulty levels, LOF achieves ROC-AUC 0.959 ± 0.031 (5-fold CV) on subtle anomalies, significantly outperforming all baselines. We frame the system as a behavioral triage tool, not a verdict engine.

1. Introduction

Online chess platforms face a persistent challenge: detecting players who use computer assistance without being detectable to casual inspection. Unlike fraud detection or network intrusion, chess integrity has no publicly available ground-truth label set — confirmed cases are treated confidentially and historical ban lists are not disclosed by platforms. This forces an **unsupervised framing**: our system identifies players whose aggregated behavioral statistics deviate from the norm established by rating-matched peers.

A player who wins at twice their Elo-implied rate, plays opening theory far deeper than expected for their rating, and finishes games in unusually few moves is statistically unusual — regardless of whether they have ever been caught. Critically, this is a **triage tool, not a verdict engine**. Flagging a player surfaces them for human review; it does not constitute an accusation. Legitimate explanations exist for every flag category: a talented beginner on a hot streak, an opening specialist, or a player whose rating has not yet caught up to their skill. The system's output is a ranked list for a human moderator, not an automated ban list.

2. Dataset and Preprocessing

2.1 Data

We use the July 2016 Lichess game export (6.25M games), sampled to 500,000 games for computational feasibility on consumer hardware. After converting to player-level records (stacking white and black perspectives) and applying a minimum-games filter (≥ 5), we obtain 17,909 unique players. We restrict to **rapid and classical time controls**: cheating via engine consultation requires enough time between moves, making faster formats (bullet, blitz) lower-signal for this task. Players with fewer than 15 games are scored by the models but excluded from ensemble flagging, as aggregated statistics are unreliable below this threshold; this retains 12,738 players (71% of the cohort) as flagging-eligible. The ensemble ultimately flags 312 players for human

review.

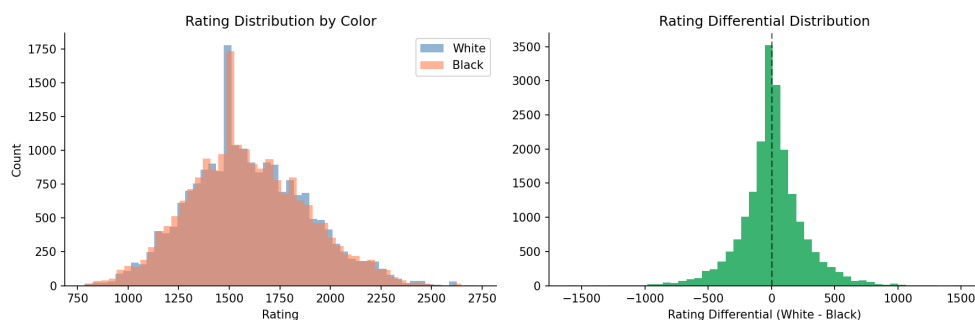


Figure 1. Rating distributions for white and black players (left) and the rating differential (right), shown from an exploratory sample of the data. Both distributions are bell-shaped and centred near 1500; the differential is tightly centred on zero, confirming balanced matchmaking across the dataset.

2.2 Preprocessing and Leakage Fix

We apply a 70/15/15 train/validation/test split *before* any normalization. `StandardScaler` is fit exclusively on training data. A subtler leakage risk was identified and corrected (**Stage 2e**): the within-rating-band z-score normalization for engine-accuracy features (ACPL-derived) was initially computed from the full player table. We recomputed per-band statistics from training players only, then propagated corrected values before refitting the scaler. Without this fix, validation and test statistics would have leaked into training-set normalization, artificially inflating reported performance.

2.3 Feature Engineering

The full feature set used by the models contains 21 features: 8 base behavioral features (Table 1) and 13 chess-accuracy features derived from Stockfish engine evaluations (ACPL variants, blunder rate, best-move rate, and phase-level accuracy signals). The 8 base features are normalized against Elo context to ensure comparability across skill levels.

Table 1. Engineered feature summary.

Feature	Description	Anomaly signal
<code>win_rate</code>	Fraction of games won	Abnormally high win rate
<code>win_rate_vs_expected</code>	Win rate minus Elo-implied rate	Consistent outperformance
<code>avg_turns</code>	Mean game length in moves	Unusually short games
<code>opening_ply_ratio</code>	Opening depth / (avg rating + 1)	Deep theory at low rating
<code>victory_efficiency</code>	$\text{avg_turns} / \text{avg_opponent_rating}$	Fast wins vs. strong opponents
<code>rating_volatility</code>	Std dev of rating over period	Sandbagging or boosting
<code>performance_vs_actual</code>	Performance rating minus avg rating	Sustained over-performance
<code>underdog_win_rate</code>	Win rate when lower-rated	Unlikely wins vs. stronger players

3. Methodology

3.1 Models

We train and evaluate seven detectors spanning complementary algorithmic families:

- **Z-Score Baseline** — flags players whose maximum absolute z-score exceeds a contamination-derived threshold; trivial baseline.
- **Isolation Forest (IF)** — isolates anomalies by random feature splits; effective on global outliers.
- **Local Outlier Factor (LOF)** — density-based; flags points whose local neighborhood density is lower than their neighbors' — suited for on-manifold anomalies.
- **One-Class SVM (OC-SVM)** — learns a kernel-based boundary enclosing normal behavior in feature space.
- **Autoencoder (AE)** — PyTorch network trained to reconstruct normal profiles; anomalies yield high reconstruction error.
- **ACPLSubAutoencoder** — diagnostic variant trained only on chess-accuracy features (ACPL, blunder rate, best-move rate). It underperformed the full Autoencoder across all benchmarks, confirming that behavioral features carry more signal than accuracy features alone for this dataset.
- **HDBSCAN** — density-based clustering applied as an outlier detector; included as an exploratory baseline to test whether natural behavioral clusters exist in the data.

3.2 Ensemble and Tuning

LOF, Autoencoder, and OC-SVM form the voting ensemble ($\geq 2/3$ votes $\rightarrow$ flagged; $3/3 \rightarrow$ high-confidence shortlist). The remaining four models were excluded from voting: IF produced too many false positives on subtle anomalies; HDBSCAN found zero meaningful clusters on this cohort (too sparse for reliable density estimation); and the ACPLSubAutoencoder was outperformed by the full Autoencoder on every benchmark. All seven models are still evaluated and reported for comparison. Hyperparameters are tuned via 10-iteration random search on the validation set using 50 injected *subtle* anomalies; the test set is never seen at this stage.

3.3 Evaluation Framework

Three synthetic injection strategies of increasing difficulty:

- ***engine_perfect*** — all features at the 99th percentile; sanity check that models function numerically.
- ***subtle*** — one-third of features perturbed by $\approx 1.5\sigma$, staying on the data manifold. Primary benchmark.
- ***realistic_cheater*** — domain-grounded profile based on published chess integrity literature [Regan(2011), Guid and Bratko(2006)]: low ACPL, elevated win rate vs. expected, deliberately normal behavioral features to evade detection.

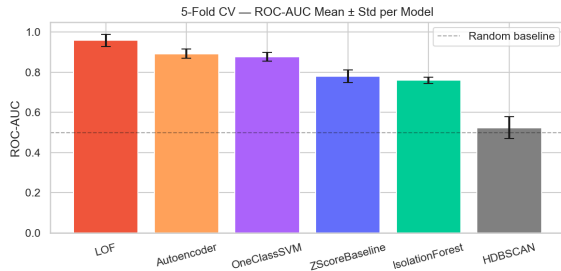
We report ROC-AUC, Average Precision, and Recall@ k (where k equals the number of injected anomalies) on both 5-fold CV and a held-out test set.

4. Results

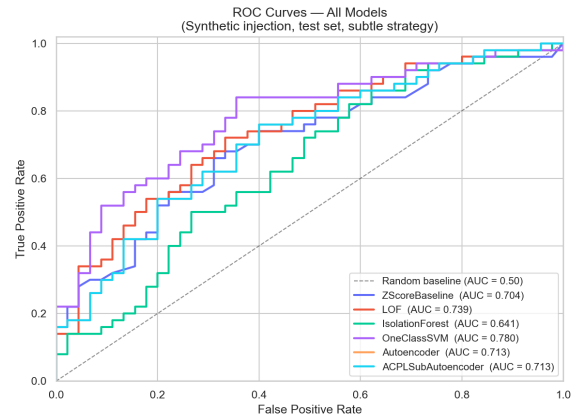
4.1 Cross-Validation

Table 2. 5-fold CV results on the *subtle* injection benchmark (50 synthetic anomalies per fold).

Model	ROC-AUC (mean $\pm$ std)	Avg Precision	Recall@ k
LOF	0.959 ± 0.031	0.626	0.604
Autoencoder	0.893 ± 0.023	0.133	0.188
One-Class SVM	0.877 ± 0.021	0.141	0.228
Z-Score Baseline	0.780 ± 0.031	0.042	0.036
Isolation Forest	0.760 ± 0.016	0.039	0.036
HDBSCAN	0.524 ± 0.054	0.017	0.564



(a) 5-fold CV ROC-AUC with standard deviation error bars. LOF leads by a large margin. HDBSCAN is near-random; its high Recall@ k in Table 2 is an artifact of memorizing one region, not general detection.



(b) ROC curves on the holdout test set (*subtle* injection). LOF (AUC = 0.971) dominates. One-Class SVM and Autoencoder track closely at AUC $\approx$ 0.89–0.91. Z-Score Baseline and Isolation Forest sit near the diagonal at high recall.

4.2 Holdout Test

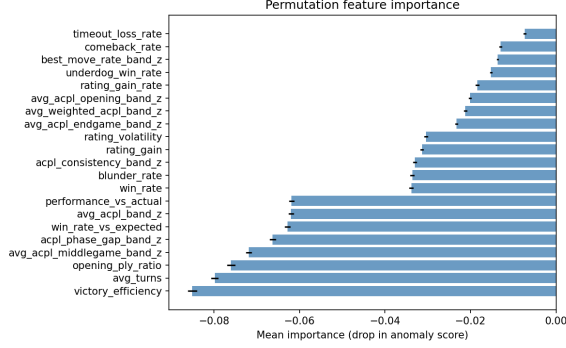
Table 3. Holdout test results (unseen 15% split, all injection strategies).

Model	Strategy	ROC-AUC	Recall@ k	Avg Precision
LOF	engine_perfect	1.000	1.00	1.000
LOF	subtle	0.971	0.62	0.720
LOF	realistic_cheater	0.740	0.01	—
Autoencoder	subtle	0.907	0.28	0.220
One-Class SVM	subtle	0.890	0.36	0.202
HDBSCAN	realistic_cheater	0.665	0.82	—
Z-Score Baseline	subtle	0.805	0.04	0.055
Isolation Forest	subtle	0.798	0.04	0.047

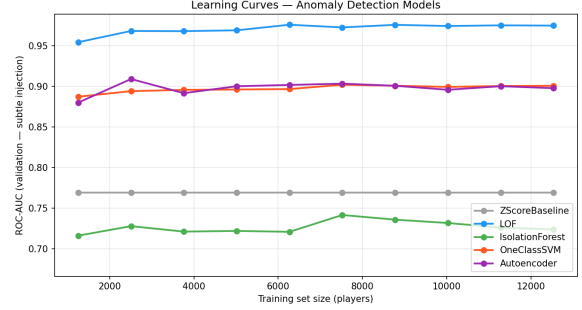
Results on the unseen test partition are consistent with cross-validation. LOF achieves AUC 0.971 and Recall@ k 0.62 on *subtle* anomalies. On the harder *realistic_cheater* benchmark,

performance degrades sharply across all models — only HDBSCAN recovers anomalies (Recall@ k 0.82) but with poor AUC (0.665), indicating it memorizes one dense region rather than learning a generalizable boundary.

4.3 Feature Importance and Signal Structure



(a) Permutation feature importance for LOF. Behavioral efficiency features dominate: `victory_efficiency`, `avg_turns`, and `opening_ply_ratio` are the three most discriminative. Raw win rate contributes the least.



(b) Learning curves by training set fraction. LOF reaches near-peak AUC at just 10% of training data, indicating a dense, learnable signal. Autoencoder and OC-SVM converge later ($\sim 40\%$), confirming LOF is particularly well-matched to this problem structure.

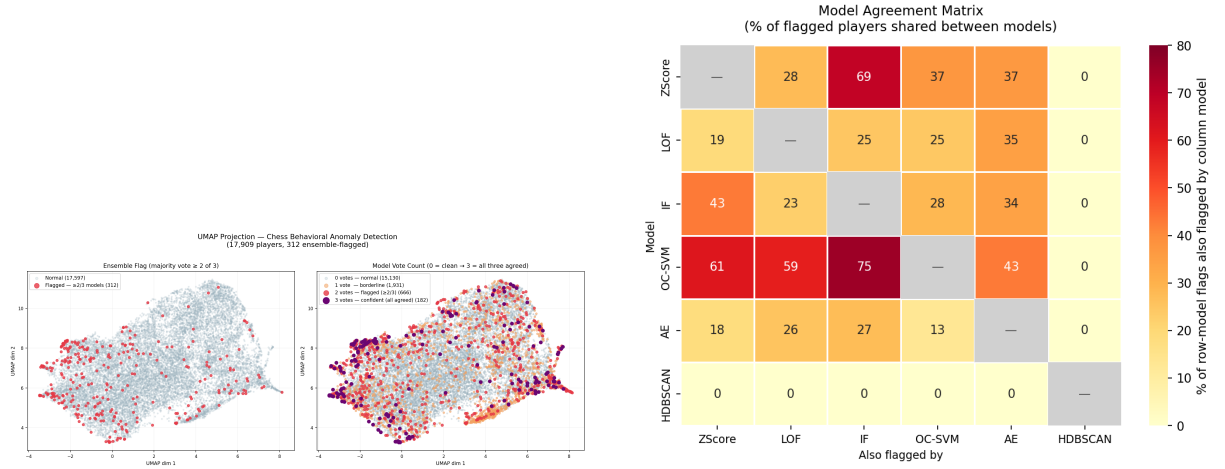
Table 4. Top permutation importances (LOF) and Welch t -test significance.

Feature	Importance (mean)	t -stat	p -value
<code>victory_efficiency</code>	-0.085	14.08	1.1×10^{-33}
<code>avg_turns</code>	-0.080	14.14	6.8×10^{-34}
<code>opening_ply_ratio</code>	-0.076	15.66	3.3×10^{-39}
<code>avg_acpl_middlegame_band_z</code>	-0.072	—	—
<code>acpl_phase_gap_band_z</code>	-0.066	—	—
<code>win_rate_vs_expected</code>	-0.063	—	—

Welch t -tests confirm that flagged players differ significantly from normal players on `opening_ply_ratio` ($t = 15.66$, $p = 3.3 \times 10^{-39}$) and `victory_efficiency` ($t = 14.08$, $p = 1.1 \times 10^{-33}$), but *not* on `blunder_rate` ($p = 0.88$). Blunder rate carries redundant signal once efficiency features are present.

5. Interpretation and Failure Analysis

We now examine what the models are detecting and where they fail:



(a) UMAP projection of the 21-dimensional feature space. Flagged players (coloured) cluster toward the periphery of the normal population (grey). Players flagged by all three ensemble members (3/3) appear furthest from the dense core, confirming the ensemble captures the most extreme behavioral deviations.

(b) Model agreement matrix across the ensemble. LOF and Autoencoder agree on only 26–35% of flagged players, confirming they capture complementary failure modes and that the ensemble is not redundant. OC-SVM overlaps more with LOF than with the Autoencoder.

5.1 What the Models Detect

The UMAP projection (Fig. 4a) is the most direct evidence that the system works as intended: flagged players are not scattered randomly through the feature space — they sit at the periphery of the normal manifold. The top features (`victory_efficiency`, `avg_turns`, `opening_ply_ratio`) tell a coherent story: suspicious players win quickly against strong opponents and display opening theory depth disproportionate to their rating. This is consistent with engine assistance but equally consistent with a legitimately skilled player whose rating has not yet caught up.

The model agreement matrix (Fig. 4b) justifies the ensemble design. If all three detectors flagged the same players, combining them would be pointless. The low overlap between LOF and the Autoencoder confirms they exploit different geometric properties of the anomaly — LOF uses local density; the Autoencoder uses reconstruction error — making the ensemble strictly better than any single model.

5.2 Why Realistic Cheaters Are Hard to Detect

The *realistic_cheater* benchmark reveals a hard ceiling: when cheaters deliberately maintain normal behavioral features and only manipulate accuracy-related signals, LOF’s $\text{Recall}@k$ collapses to 0.01 (Table 3). The missing signal is `move_time_cv` (coefficient of variation in per-move think times) — engine-assisted players exhibit unnaturally consistent decision times. This feature requires clock-annotated game data not present in the July 2016 Lichess export; any dataset from 2017 onwards includes clock data and would likely close this gap substantially.

6. Reflection and Limitations

Core limitation. All metrics are computed against synthetic anomalies we designed ourselves. The AUC of 0.971 measures how well LOF detects the *kind* of anomaly we constructed — not performance on confirmed real-world cases, which do not exist in any public dataset. This is not a gap in methodology; it is the honest reality of the problem domain.

What worked. LOF emerged as a stable, data-efficient winner — near-peak AUC at just 10% of training data (Fig. 3b). The three-strategy injection framework gave an honest picture of difficulty scaling. The Stage 2e leakage correction fixed a subtle risk that would otherwise have inflated reported training-set statistics.

What did not work. HDBSCAN failed entirely on this cohort — too sparse for cluster formation. The *realistic_cheater* benchmark exposed a hard ceiling that our behavioral feature set cannot currently cross without clock-annotated data.

Ethics. No player is named in any output. All flagged players are described as warranting review, not confirmed bad actors. The system’s output is a ranked list for a human moderator — not an automated ban list.

7. Group Collaboration

We split responsibilities across four roles: **Christoph** — Evaluation & Validation; **Kiril** — Modelling & Pipeline; **Ali** — Data & Preprocessing; **Georgy** — Features & Reporting. All members contributed to the poster and presentation. Full commit history is traceable on GitHub.

8. Acknowledgments

AI writing tools were used for editorial revision of this document.

References

- [Breunig et al.(2000)] Breunig, M. M., Kriegel, H.-P., Ng, R. T., and Sander, J. (2000). LOF: Identifying density-based local outliers. *ACM SIGMOD Record*, 29(2):93–104.
- [Guid and Bratko(2006)] Guid, M. and Bratko, I. (2006). Computer analysis of world chess champions. *ICGA Journal*, 29(2):65–73.
- [McInnes et al.(2018)] McInnes, L., Healy, J., and Melville, J. (2018). UMAP: Uniform manifold approximation and projection. *arXiv:1802.03426*.
- [Oxera(2023)] Oxera (2023). Fair play in online chess: An economic perspective. Oxera Consulting.
- [Pedregosa et al.(2011)] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830.
- [Regan(2011)] Regan, K. W. (2011). Intrinsic chess ratings. *AAAI Workshop on Machine Learning and Games*.